

Java's History

- Java was created by **James Gosling** and his team at **Sun Microsystems** in 1991, originally called "**Oak**" (named after an oak tree outside Gosling's office), later renamed **Java** in 1995.
- It was designed for embedded consumer electronics, but found its real success with the rise of the **Internet** because of its portability.
- Sun Microsystems was later acquired by **Oracle Corporation** in 2010, which now maintains Java.
- Java derives its syntax from **C/C++** but removes complex features like pointers, multiple inheritance (of classes), and operator overloading.

Creation of Java

Java was built around five core design goals:

1. Simple, object-oriented, and familiar.
2. Robust and secure.
3. Architecture-neutral and portable.
4. High performance.
5. Interpreted, threaded, and dynamic.

Internet & Java

- Java was the first language designed for **distributed environments** like the Internet.
- It handles **TCP/IP protocols** natively, allowing remote resource access as easily as local files.
- Two important security mechanisms: **Applet** (small Java programs that run inside a browser) and **Byte-code** based execution which sandbox untrusted code.

Byte-code

- Java source code (`.java`) is compiled by `javac` into **byte-code** (`.class` files), not directly into machine code.
- Byte-code is a highly optimized set of instructions executed by the **Java Virtual Machine (JVM)**.
- This is the core of Java's "**Write Once, Run Anywhere**" (**WORA**) philosophy — the same `.class` file runs on any platform that has a compatible JVM.

Features of Java

Feature	Explanation
Simple	Syntax close to C++ but without pointers, operator overloading, multiple inheritance
Object-Oriented	Almost everything is an object (except primitive types)
Platform Independent	Byte-code runs on any JVM
Secure	No explicit pointers; runs in a sandboxed environment
Robust	Strong memory management, exception handling, automatic garbage collection
Architecture-Neutral	No implementation-dependent data sizes
Portable	Same byte-code behavior across platforms
Multithreaded	Built-in support for concurrent execution
Interpreted & High Performance	JVM interprets byte-code; JIT compiler boosts speed
Distributed	Designed to work over networks
Dynamic	Classes loaded as needed at runtime

Questions

1. Name any two features of Java that make it "architecture-neutral." (2 Marks)

Java is called **architecture-neutral** because its programs can run on any hardware platform without modification.

Two features are:

1. **Bytecode Generation:** Java source code is compiled into platform-independent bytecode instead of machine code.
2. **Java Virtual Machine (JVM):** The JVM converts the bytecode into machine code specific to the host system, allowing the same program to run on different architectures.

2. What is the purpose of the `final` keyword when applied to a method? (2 Marks)

When the `final` keyword is applied to a method, it **prevents the method from being overridden** by subclasses.

This is used when the programmer wants to preserve the original implementation of the method.

Example

```
class Animal {
    final void sound() {
        System.out.println("Animal Sound");
    }
}

class Dog extends Animal {
    // Error: Cannot override final method
    // void sound() { }
}
```

3. Explain operator precedence in Java with an example expression evaluated step by step. (6 Marks)

Definition

Operator precedence is the order in which Java evaluates operators in an expression. Operators with higher precedence are evaluated before operators with

lower precedence. If operators have the same precedence, associativity rules are followed.

Common Precedence (Highest to Lowest)

1. `()` (Parentheses)
2. Unary (`++` , `--` , `!`)
3. `*` , `/` , `%`
4. `+` , `-`
5. Relational (`<` , `>` , `<=` , `>=`)
6. Equality (`==` , `!=`)
7. Logical AND (`&&`)
8. Logical OR (`||`)
9. Assignment (`=`)

Example

```
int result = 10 + 5 * 2 - 8 / 4;
```

Expression:

```
10 + 5 * 2 - 8 / 4
```

Step 1: Multiplication

```
10 + 10 - 8 / 4
```

Step 2: Division

```
10 + 10 - 2
```

Step 3: Addition

20 - 2

Step 4: Subtraction

18

Output

Result = 18

4. What is the Java Virtual Machine? Explain in detail how Java achieves platform independence using byte-code and JVM, with a labeled diagram of the compilation and execution process. (10 Marks)

Definition

The **Java Virtual Machine (JVM)** is a software-based virtual machine that executes Java bytecode. It acts as an interface between Java programs and the operating system.

Platform Independence

Java is known as "**Write Once, Run Anywhere (WORA)**" because the same compiled Java program can run on any operating system that has a compatible JVM.

Working Process

Step 1: Writing the Program

The programmer writes the source code in a file with the `.java` extension.

Example:

```
Hello.java
```

Step 2: Compilation

The Java Compiler (`javac`) compiles the source code into **bytecode**.

```
Hello.java
  |
  | javac
  |
Hello.class
```

The `.class` file contains **bytecode**, which is platform independent.

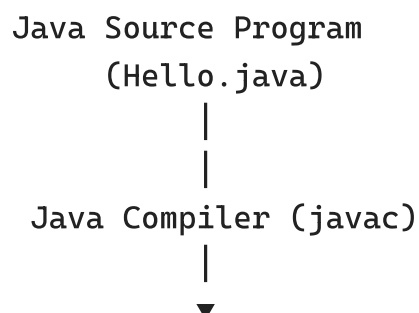
Step 3: JVM Execution

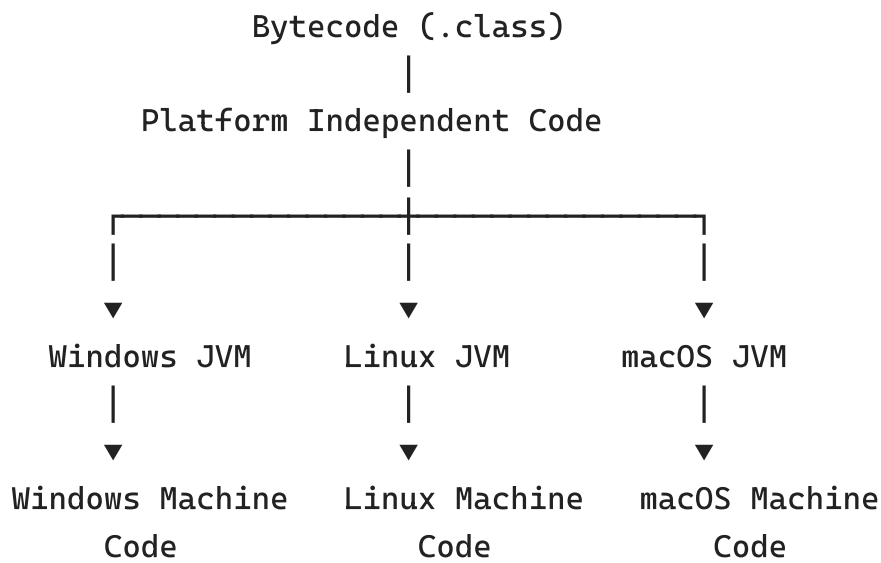
The JVM loads the bytecode and performs the following tasks:

- Class Loading
- Bytecode Verification
- Memory Management
- Garbage Collection
- Execution through the Interpreter or JIT (Just-In-Time) Compiler

Finally, the JVM converts the bytecode into machine code for the specific operating system.

Labeled Diagram





Advantages of JVM

- Provides platform independence.
- Executes bytecode on any operating system.
- Performs automatic memory management.
- Supports garbage collection.
- Provides security through bytecode verification.
- Improves performance using the JIT compiler.

Advantages of Bytecode

- Platform independent.
- Secure and portable.
- Can run on any system with a JVM.
- Supports the "Write Once, Run Anywhere" concept.